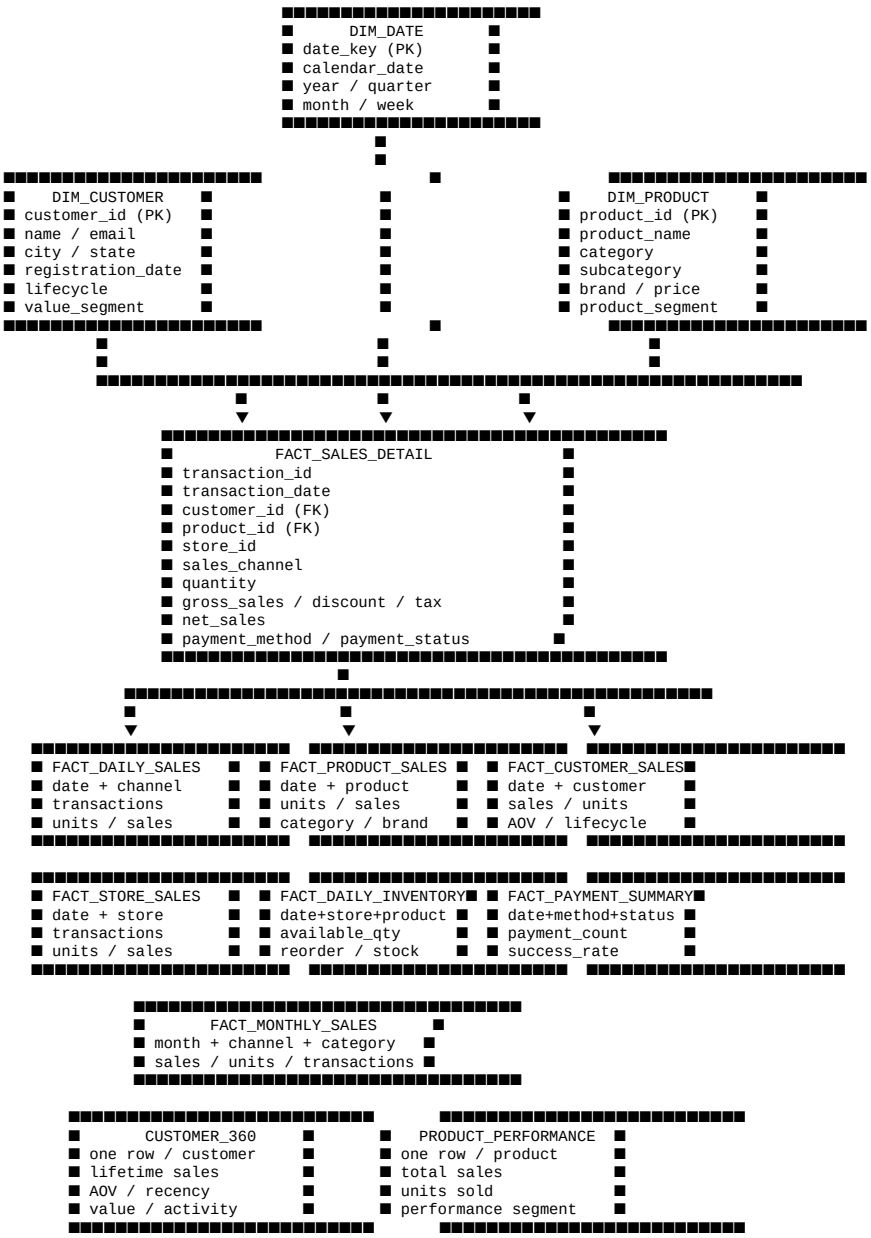
Retail Analytics — Gold Layer

Gold Layer Model + Business Insight SQL + Databricks PySpark

Updated reference document for the Retail Analytics Gold layer. It adds a clear dimensional model diagram and a practical SQL library for business analysis.

1. Clear Gold Layer Model



Modeling approach: the Gold layer contains reusable dimensions, detailed/enriched sales facts, performance aggregates, and purpose-built analytical marts. BI dashboards can primarily consume the Gold aggregate tables and Customer 360/Product Performance tables, while analysts can drill down to FACT_SALES_DETAIL.

2. SQL Queries for Business Insights

The following queries assume the Gold schema is **retail_catalog.gold_retail**. They are intended for Databricks SQL and can be used directly in dashboards, SQL Warehouses, or notebooks.

2.1 Executive Sales KPIs

```
SELECT
    SUM(net_sales) AS total_sales,
    SUM(units_sold) AS total_units,
    SUM(transaction_count) AS total_transactions,
    ROUND(SUM(net_sales) / NULLIF(SUM(transaction_count), 0), 2) AS avg_transaction_value
FROM retail_catalog.gold_retail.fact_daily_sales;
```

2.2 Daily Sales Trend

```
SELECT
    transaction_date,
    SUM(net_sales) AS sales,
    SUM(units_sold) AS units_sold,
    SUM(transaction_count) AS transactions
FROM retail_catalog.gold_retail.fact_daily_sales
GROUP BY transaction_date
ORDER BY transaction_date;
```

2.3 Monthly Sales Trend

```
SELECT
    sales_year,
    sales_month,
    sales_year_month,
    SUM(net_sales) AS net_sales,
    SUM(units_sold) AS units_sold,
    SUM(transaction_count) AS transactions
FROM retail_catalog.gold_retail.fact_monthly_sales
GROUP BY sales_year, sales_month, sales_year_month
ORDER BY sales_year, sales_month;
```

2.4 Sales by Channel

```
SELECT
    sales_channel,
    SUM(net_sales) AS net_sales,
    SUM(units_sold) AS units_sold,
    SUM(transaction_count) AS transactions,
    ROUND(SUM(net_sales) / NULLIF(SUM(transaction_count), 0), 2) AS avg_order_value
FROM retail_catalog.gold_retail.fact_daily_sales
GROUP BY sales_channel
ORDER BY net_sales DESC;
```

2.5 Sales by Category

```
SELECT
    category,
    SUM(net_sales) AS net_sales,
    SUM(units_sold) AS units_sold,
    SUM(transaction_count) AS transactions
FROM retail_catalog.gold_retail.fact_product_sales
GROUP BY category
ORDER BY net_sales DESC;
```

2.6 Top 10 Products by Revenue

```
SELECT
    product_id,
    product_name,
    category,
    brand,
    SUM(net_sales) AS total_sales,
    SUM(units_sold) AS units_sold
FROM retail_catalog.gold_retail.fact_product_sales
GROUP BY product_id, product_name, category, brand
ORDER BY total_sales DESC
LIMIT 10;
```

2.7 Bottom 10 Products by Revenue

```
SELECT
    product_id,
    product_name,
    category,
    SUM(net_sales) AS total_sales,
    SUM(units_sold) AS units_sold
FROM retail_catalog.gold_retail.fact_product_sales
GROUP BY product_id, product_name, category
ORDER BY total_sales ASC
LIMIT 10;
```

2.8 Customer Value Segmentation

```
SELECT
    customer_value_segment,
    COUNT(*) AS customers,
    SUM(lifetime_sales) AS segment_sales,
    AVG(lifetime_sales) AS avg_customer_sales,
    AVG(total_transactions) AS avg_transactions
FROM retail_catalog.gold_retail.customer_360
GROUP BY customer_value_segment
ORDER BY segment_sales DESC;
```

2.9 Top 20 Customers

```
SELECT
    customer_id,
    customer_name,
    city,
    state,
    lifetime_sales,
    total_transactions,
    average_order_value,
    last_purchase_date,
    customer_value_segment
FROM retail_catalog.gold_retail.customer_360
ORDER BY lifetime_sales DESC
LIMIT 20;
```

2.10 At-Risk Customers

```
SELECT
    customer_id,
    customer_name,
    lifetime_sales,
    total_transactions,
    last_purchase_date,
    days_since_last_purchase,
    customer_value_segment
FROM retail_catalog.gold_retail.customer_360
WHERE customer_activity_status = 'AT_RISK'
ORDER BY lifetime_sales DESC;
```

2.11 Dormant High-Value Customers

```
SELECT
    customer_id,
    customer_name,
    lifetime_sales,
    total_transactions,
    last_purchase_date,
    days_since_last_purchase
FROM retail_catalog.gold_retail.customer_360
WHERE customer_activity_status = 'DORMANT'
  AND customer_value_segment IN ('GOLD', 'PLATINUM')
ORDER BY lifetime_sales DESC;
```

2.12 Store Performance

```
SELECT
    store_id,
    SUM(net_sales) AS total_sales,
    SUM(units_sold) AS units_sold,
    SUM(transaction_count) AS transactions,
    ROUND(SUM(net_sales) / NULLIF(SUM(transaction_count), 0), 2) AS avg_transaction_value
FROM retail_catalog.gold_retail.fact_store_sales
GROUP BY store_id
ORDER BY total_sales DESC;
```

2.13 Inventory Reorder Candidates

```
SELECT
    inventory_date,
    store_id,
    product_id,
    product_name,
    category,
    available_qty,
    reorder_level,
    safety_stock,
    stock_status,
    reorder_required
FROM retail_catalog.gold_retail.fact_daily_inventory
WHERE reorder_required = 1
ORDER BY inventory_date DESC, available_qty ASC;
```

2.14 Low Stock Products

```
SELECT
    product_id,
    product_name,
    category,
    SUM(CASE WHEN stock_status = 'LOW_STOCK' THEN 1 ELSE 0 END) AS low_stock_days,
    MIN(available_qty) AS minimum_available_qty
FROM retail_catalog.gold_retail.fact_daily_inventory
GROUP BY product_id, product_name, category
HAVING low_stock_days > 0
ORDER BY low_stock_days DESC, minimum_available_qty ASC;
```

2.15 Payment Success Rate

```
SELECT
    payment_method,
    SUM(payment_count) AS payment_count,
    SUM(successful_payment_count) AS successful_payments,
    SUM(failed_payment_count) AS failed_payments,
    ROUND(
        SUM(successful_payment_count) * 100.0 /
        NULLIF(SUM(payment_count), 0), 2
    ) AS success_rate_pct
FROM retail_catalog.gold_retail.fact_payment_summary
GROUP BY payment_method
ORDER BY success_rate_pct DESC;
```

2.16 Failed Payments

```
SELECT
    payment_method,
    SUM(failed_payment_count) AS failed_payments,
    SUM(payment_amount) AS payment_amount
```

```
FROM retail_catalog.gold_retail.fact_payment_summary
WHERE payment_status <> 'SUCCESS'
GROUP BY payment_method
ORDER BY failed_payments DESC;
```

2.17 Month-over-Month Sales Growth

```
WITH monthly AS (
    SELECT
        sales_year,
        sales_month,
        sales_year_month,
        SUM(net_sales) AS net_sales
    FROM retail_catalog.gold_retail.fact_monthly_sales
    GROUP BY sales_year, sales_month, sales_year_month
)
SELECT
    sales_year_month,
    net_sales,
    LAG(net_sales) OVER (
        ORDER BY sales_year, sales_month
    ) AS previous_month_sales,
    ROUND(
        (net_sales - LAG(net_sales) OVER (ORDER BY sales_year, sales_month))
        * 100.0 /
        NULLIF(LAG(net_sales) OVER (ORDER BY sales_year, sales_month), 0),
        2
    ) AS mom_growth_pct
FROM monthly
ORDER BY sales_year, sales_month;
```

2.18 Category Growth by Month

```
WITH monthly AS (
    SELECT
        sales_year,
        sales_month,
        sales_year_month,
        category,
        SUM(net_sales) AS net_sales
    FROM retail_catalog.gold_retail.fact_monthly_sales
    GROUP BY sales_year, sales_month, sales_year_month, category
)
SELECT
    sales_year_month,
    category,
    net_sales,
    LAG(net_sales) OVER (
        PARTITION BY category
        ORDER BY sales_year, sales_month
    ) AS previous_month_sales,
    ROUND(
        (net_sales - LAG(net_sales) OVER (
            PARTITION BY category ORDER BY sales_year, sales_month
        )) * 100.0 /
        NULLIF(LAG(net_sales) OVER (
            PARTITION BY category ORDER BY sales_year, sales_month
        ), 0),
        2
    ) AS mom_growth_pct
FROM monthly
ORDER BY sales_year, sales_month, category;
```

2.19 Product Performance Ranking

```
SELECT
    product_id,
    product_name,
    category,
    total_sales,
    total_units_sold,
    total_transactions,
    average_selling_price,
    product_performance,
    DENSE_RANK() OVER (ORDER BY total_sales DESC) AS sales_rank
FROM retail_catalog.gold_retail.product_performance
ORDER BY sales_rank;
```

2.20 Customer Purchase Frequency

```
SELECT
    CASE
        WHEN total_transactions = 0 THEN 'NO PURCHASE'
        WHEN total_transactions = 1 THEN 'ONE-TIME'
        WHEN total_transactions BETWEEN 2 AND 5 THEN 'OCCASIONAL'
        WHEN total_transactions BETWEEN 6 AND 12 THEN 'REGULAR'
        ELSE 'FREQUENT'
    END AS purchase_frequency,
    COUNT(*) AS customers,
    SUM(lifetime_sales) AS sales
FROM retail_catalog.gold_retail.customer_360
GROUP BY
    CASE
        WHEN total_transactions = 0 THEN 'NO PURCHASE'
        WHEN total_transactions = 1 THEN 'ONE-TIME'
        WHEN total_transactions BETWEEN 2 AND 5 THEN 'OCCASIONAL'
        WHEN total_transactions BETWEEN 6 AND 12 THEN 'REGULAR'
        ELSE 'FREQUENT'
    END
ORDER BY sales DESC;
```

2.21 Average Basket Size by Channel

```
SELECT
    sales_channel,
    ROUND(
        SUM(units_sold) * 1.0 /
```

```
        NULLIF(SUM(transaction_count), 0), 2
    ) AS avg_units_per_transaction,
    ROUND(
        SUM(net_sales) * 1.0 /
        NULLIF(SUM(transaction_count), 0), 2
    ) AS avg_transaction_value
FROM retail_catalog.gold_retail.fact_daily_sales
GROUP BY sales_channel
ORDER BY avg_transaction_value DESC;
```

2.22 Sales by Brand

```
SELECT
    brand,
    SUM(net_sales) AS total_sales,
    SUM(units_sold) AS units_sold,
    SUM(transaction_count) AS transactions
FROM retail_catalog.gold_retail.fact_product_sales
GROUP BY brand
ORDER BY total_sales DESC;
```

2.23 Weekend vs Weekday Sales

```
SELECT
    CASE
        WHEN d.is_weekend THEN 'WEEKEND'
        ELSE 'WEEKDAY'
    END AS day_type,
    SUM(s.net_sales) AS net_sales,
    SUM(s.units_sold) AS units_sold,
    SUM(s.transaction_count) AS transactions
FROM retail_catalog.gold_retail.fact_daily_sales s
JOIN retail_catalog.gold_retail.dim_date d
  ON s.transaction_date = d.calendar_date
GROUP BY CASE WHEN d.is_weekend THEN 'WEEKEND' ELSE 'WEEKDAY' END
ORDER BY net_sales DESC;
```

2.24 Customer and Product Cross Analysis

```
SELECT
    p.category,
    c.customer_value_segment,
    SUM(s.net_sales) AS net_sales,
    SUM(s.units_sold) AS units_sold,
    COUNT(DISTINCT s.customer_id) AS customers
FROM retail_catalog.gold_retail.fact_sales_detail s
JOIN retail_catalog.gold_retail.dim_customer c
  ON s.customer_id = c.customer_id
JOIN retail_catalog.gold_retail.dim_product p
  ON s.product_id = p.product_id
GROUP BY p.category, c.customer_value_segment
ORDER BY net_sales DESC;
```

2.25 Daily Sales Anomaly Candidates

```
WITH x AS (
    SELECT
        transaction_date,
        SUM(net_sales) AS net_sales
    FROM retail_catalog.gold_retail.fact_daily_sales
    GROUP BY transaction_date
),
stats AS (
    SELECT
        AVG(net_sales) AS avg_sales,
        STDDEV_POP(net_sales) AS std_sales
    FROM x
)
SELECT
    x.transaction_date,
    x.net_sales,
    ROUND((x.net_sales - stats.avg_sales) / NULLIF(stats.std_sales,0), 2) AS z_score
FROM x
CROSS JOIN stats
WHERE ABS((x.net_sales - stats.avg_sales) / NULLIF(stats.std_sales,0)) >= 2
ORDER BY ABS(z_score) DESC;
```

3. Gold Layer Consumption Guide

```
EXECUTIVE DASHBOARD
  ■■■ fact_daily_sales
  ■■■ fact_monthly_sales
  ■■■ fact_store_sales

CUSTOMER ANALYTICS
  ■■■ customer_360
  ■■■ fact_customer_sales

PRODUCT ANALYTICS
  ■■■ product_performance
  ■■■ fact_product_sales

INVENTORY / SUPPLY
  ■■■ fact_daily_inventory

PAYMENT ANALYTICS
  ■■■ fact_payment_summary

DETAILED DRILL-DOWN
  ■■■ fact_sales_detail
```

Important: the SQL queries use the column names defined by the Gold notebook. If the final Silver-to-Gold implementation uses different names, update the SQL accordingly.

4. Production Recommendations

• Use Delta MERGE/incremental processing rather than full overwrite for production pipelines.

• Parameterize business thresholds such as customer value bands and product performance bands.

• Use date-based incremental processing for daily and monthly fact tables.

• Add data-quality checks for duplicates, null keys, negative amounts, and referential integrity.

• Expose Gold tables through Databricks SQL / BI tools for governed reporting.

• Keep FACT_SALES_DETAIL for drill-down and aggregate Gold facts for dashboard performance.